



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Desktop Widget Settings

A dedicated configuration interface for their desktop widget. Unlike the general [Settings UI](#) which configures plugin-wide settings. Desktop Widget Settings operate on **per-instance widget data**, the position, scale, and custom properties stored for each individual widget on the desktop.

Basic Structure

A Desktop Widget Settings component is a QML file that receives a `pluginApi` and a `widgetSettings` object:

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Widgets

ColumnLayout {
    id: root
    spacing: Style.marginM

    // Plugin API (injected by the settings dialog system)
    property var pluginApi: null

    // Widget settings object (injected by the settings dialog system)
    property var widgetSettings: null

    // Local state - initialize from widgetSettings.data with metadata fallback
    property bool valueShowBackground:
        widgetSettings?.data?.showBackground ??
        widgetSettings?.metadata?.showBackground ??
        true

    // Your settings controls here

    NToggle {
        Layout.fillWidth: true
        label: "Show Background"
        description: "Display the widget background container"
        checked: root.valueShowBackground
        onToggled: checked ⇒ {
            root.valueShowBackground = checked;
            saveSettings();
        }
    }

    function saveSettings() {
        if (!widgetSettings) return;
        widgetSettings.data.showBackground = root.valueShowBackground;
        widgetSettings.save();
    }
}
```

```
}
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

HOW IT WORKS

When a user right-clicks a desktop widget and selects **Widget Settings** (or clicks the gear icon in the Desktop Widgets settings tab), Noctalia loads your `desktopWidgetSettings` component inside a `DesktopWidgetSettingsDialog`. The dialog injects two properties into your component:

pluginApi

The standard [Plugin API](#) object. Use it to access plugin-wide settings, translations, manifest data, and more.

```
property var pluginApi: null
```

widgetSettings

An object that provides access to the current widget instance's data:

```
property var widgetSettings: null
```

The `widgetSettings` object contains two variables and one function:

data

- **Type:** object
- **Read-only:** no
- **Description:** The current widget instance's stored settings (position, scale, and any custom keys you've added). Modify this object directly.

metadata

- **Type:** object

- **Read-only:** yes
- **Description:** Read-only default metadata registered by the DesktopWidgetRegistry for

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

save()

- **Type:** function
- **Signature:** function save(): void
- **Description:** Call this to persist the current widgetSettings.data to disk.

Caution

Always call widgetSettings.save() after modifying widgetSettings.data to persist changes. The dialog also calls saveSettings() on your component when closing, so implement that function to ensure any pending changes are saved.

Manifest Entry

Register the desktop widget settings in your plugin's manifest.json:

```
{
  "id": "my-plugin",
  "name": "My Plugin",
  "version": "1.0.0",
  "entryPoints": {
    "desktopWidget": "DesktopWidget.qml",
    "desktopWidgetSettings": "DesktopWidgetSettings.qml"
  },
  "metadata": {
    "defaultSettings": {
      "message": "Hello World",
      "enabled": true,
      "count": 0
    }
  }
}
```

Note

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Note

When both `desktopWidgetSettings` and `settings` are defined, the desktop widget settings menu will load `desktopWidgetSettings`. The general settings entry point is used for the plugin-wide configuration accessed from **Plugins > Your Plugin > Configure**.

Local State Pattern

Follows the same local state pattern used in the Settings UI. Initialize local properties from `widgetSettings.data` with fallbacks to `widgetSettings.metadata`, `pluginApi.pluginSettings` or `pluginApi.manifest.metadata.defaultSettings`:

```
ColumnLayout {  
  id: root
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
property var pluginApi: null  
property var widgetSettings: null  
  
// Local state with metadata fallback  
property bool valueShowBackground:  
  widgetSettings?.data?.showBackground ??  
  widgetSettings?.metadata?.showBackground ??  
  true  
  
property bool valueRoundedCorners:  
  widgetSettings?.data?.roundedCorners ??  
  widgetSettings?.metadata?.roundedCorners ??  
  true  
  
property string valueLayout:  
  widgetSettings?.data?.layout ??  
  pluginApi?.manifest?.metadata?.defaultSettings?.layout ??  
  "side"  
  
function save() {  
  if (!widgetSettings) return;  
  widgetSettings.data.showBackground = root.valueShowBackground;  
  widgetSettings.data.roundedCorners = root.valueRoundedCorners;  
  widgetSettings.data.layout = root.valueLayout;  
  widgetSettings.save();  
}  
}
```

Tip

Unlike the general Settings UI where `saveSettings()` is called by the dialog on “Save”, Desktop Widget Settings use a **live-save** pattern. Call `widgetSettings.save()` whenever a value changes.

settingsChanged signal

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

signal. The settings dialog listens for this signal (debounced at 150ms) and handles persistence automatically. You can also emit settingsSaved for immediate (non-debounced) persistence:

```
ColumnLayout {
  id: root

  property var pluginApi: null
  property var widgetSettings: null

  signal settingsChanged(var settings)

  property bool valueShowBackground:
    widgetSettings?.data?.showBackground ??
    widgetSettings?.metadata?.showBackground ??
    true

  function save() {
    var settings = Object.assign({}, widgetSettings?.data || {});
    settings.showBackground = root.valueShowBackground;
    settingsChanged(settings);
  }

  NToggle {
    Layout.fillWidth: true
    label: "Show Background"
    checked: root.valueShowBackground
    onToggled: checked ⇒ {
      root.valueShowBackground = checked;
      root.save();
    }
  }
}
```

 Note

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Complete Example

Here's a complete desktop widget settings example:


```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Widgets

ColumnLayout {
    id: root

    property var pluginApi: null
    property var widgetSettings: null

    // Local state
    property string editMessage:
        widgetSettings?.data?.message ??
        pluginApi?.manifest?.metadata?.defaultSettings?.message ??
        "Hello World"

    property color editBgColor:
        widgetSettings?.data?.backgroundColor ??
        pluginApi?.manifest?.metadata?.defaultSettings?.backgroundColor ??
        "#A9AEFE"

    property bool editShowIcon:
        widgetSettings?.data?.showIcon ??
        pluginApi?.manifest?.metadata?.defaultSettings?.showIcon ??
        true

    property int editRefreshRate:
        widgetSettings?.data?.refreshRate ??
        pluginApi?.manifest?.metadata?.defaultSettings?.refreshRate ??
        30

    // Have a toggle for the built-in showBackground value as well
    property bool showBackground:
        widgetSettings?.data?.showBackground ??
        widgetSettings?.metadata?.showBackground ??
        true

    spacing: Style.marginM
```

Component.onCompleted: {

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Text input
NTextInput {
  Layout.fillWidth: true
  label: "Display Message"
  description: "The message shown in the bar widget"
  placeholderText: "Enter a message..."
  text: root.editMessage
  onTextChanged: {
    root.editMessage = text;
    root.saveSettings();
  }
}

// Color picker
ColumnLayout {
  Layout.fillWidth: true
  spacing: Style.marginS

  NLabel {
    label: "Background Color"
    description: "Widget background color"
  }

  NColorPicker {
    Layout.preferredWidth: Style.sliderWidth
    Layout.preferredHeight: Style.baseWidgetSize
    selectedColor: root.editBgColor
    onColorSelected: function(color) {
      root.editBgColor = color;
      root.saveSettings();
    }
  }
}

NDivider {
```

Layout.fillWidth: **true**

Layout.topMargin: **Style.marginS**

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Toggle
NToggle {
  Layout.fillWidth: true
  label: "Show Icon"
  description: "Display an icon next to the message"
  checked: root.editShowIcon
  onToggled: checked ⇒ {
    root.editShowIcon = checked;
    root.saveSettings();
  }
}

// Slider with label
ColumnLayout {
  Layout.fillWidth: true
  spacing: Style.marginS

  NLabel {
    label: "Refresh Rate"
    description: "Update interval in seconds: " + root.editRefreshRate
  }

  NSlider {
    Layout.fillWidth: true
    from: 5
    to: 120
    stepSize: 5
    value: root.editRefreshRate
    onValueChanged: {
      root.editRefreshRate = value;
      root.saveSettings();
    }
  }
}
```

```
// Show background toggle
```

```
NToggle {
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
description: "Show the background of the desktop widget"
```

```
checked: root.showBackground
```

```
onToggled: checked ⇒ {
```

```
  root.showBackground = checked;
```

```
  root.saveSettings();
```

```
}
```

```
}
```

```
// Preview section
```

```
NDivider {
```

```
  Layout.fillWidth: true
```

```
  Layout.topMargin: Style.marginS
```

```
  Layout.bottomMargin: Style.marginS
```

```
}
```

```
NLabel {
```

```
  label: "Preview"
```

```
}
```

```
Rectangle {
```

```
  Layout.fillWidth: true
```

```
  Layout.preferredHeight: 50
```

```
  color: root.editBgColor
```

```
  radius: Style.radiusM
```

```
RowLayout {
```

```
  anchors.centerIn: parent
```

```
  spacing: Style.marginS
```

```
  NIcon {
```

```
    icon: "heart"
```

```
    visible: root.editShowIcon
```

```
    color: Color.mOnPrimary
```

```
  }
```

```
  NText {
```

```
    text: root.editMessage
```

color: `Color.mOnPrimary`

pointSize: `Style.fontSizeM`

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
}

// Save function - called by the dialog
function saveSettings() {
  if (!widgetSettings || !widgetSettings.data) {
    Logger.e("MyPlugin", "Cannot save: widgetSettings is null")
    return
  }

  widgetSettings.data.message = root.editMessage
  widgetSettings.data.backgroundColor = root.editBgColor.toString()
  widgetSettings.data.showIcon = root.editShowIcon
  widgetSettings.data.refreshRate = root.editRefreshRate
  widgetSettings.data.showBackground = root.showBackground;

  widgetSettings.save();

  Logger.i("MyPlugin", "Settings saved successfully")
}
}
```

Best Practices

1. **Use for per-instance options only:** Desktop widget settings should control appearance and behavior of a specific widget instance (background, colors, layout). Plugin-wide settings (API keys, global toggles) belong in the general settings entry point.
2. **Provide metadata fallbacks:** Always fall back to `widgetSettings.metadata` values so widgets have sensible defaults before the user configures them.
3. **Save on change:** Use the live-save pattern, call `widgetSettings.save()` immediately when values change. The system debounces writes automatically.
4. **Use Noctalia widgets:** Use `NToggle`, `NDropdown`, `NTextInput`, and other Noctalia components for a consistent look.

See Also

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

-
- [Bar widget Development](#) - Create bar widgets
 - [Desktop Widget Development](#) - Create desktop widgets
 - [Panel Development](#) - Create overlay panels
 - [Plugin API](#) - Full API reference
 - [Manifest Reference](#) - Plugin configuration